Unidad 1 / Escenario 1

Lectura fundamental

Características fundamentales de los procesos de desarrollo de *software*

Contenido

1 Introducción al proceso de *software* personal. ¿Qué es el proceso de *software* personal?

2 Métricas vs Indicadores de desarrollo de *software*

3 Formatos de registro y medición

Palabras clave: proceso de *software* personal, métricas, indicadores, formato PSP, construcción de *software*.

1. Introducción al proceso de *software* personal ¿Qué es el proceso de *software* personal?

Durante mucho tiempo se han venido diseñando estrategias para caracterizar la manera como un programador realiza su trabajo, sin embargo, existen varios obstáculos. Para identificar algunos de estos obstáculos es importante pensar en el desarrollo de *software* como un pequeño proyecto en el que usualmente cada desarrollo es diferente a cualquiera de los anteriores.

La diferencia entre cada proyecto se debe a múltiples factores que no siempre están juntos y aparecen de manera esporádica o recurrente: todas las veces no se parte del mismo punto, fase o etapa, el cliente es distinto, el desarrollador es diferente o no tiene experiencia, no conoce la forma en el que el mismo trabaja, hay dificultades para desarrollar el proyecto, no se conocen o dominan las librerías a utilizar, no se tienen los conocimientos o experiencias en el tema, no se domina el lenguaje de programación y muchos más...

¿Sabía qué...?

Según CAOS REPORT, para el 2016, cerca del 70% de los proyectos de TI fracasa o se modifican.
Una de las causas para no cumplir los objetivos es que no se conoce el proceso personal de desarrollo de *software*.

Surge entonces la pregunta ¿Cómo se hace para caracterizar, predecir y controlar un fenómeno tan cambiante? Pues bien, se puede lograr partiendo de la forma como naturalmente se construye *software*, del estudio de los hábitos y la manera como se utiliza el tiempo, de los conocimientos que tengamos en desarrollo y el esfuerzo que implique cada proyecto.

El proceso de desarrollo de *software* tiene usualmente asociadas fases o partes o etapas y en los casos en los que no ocurra, podemos dividirlas en ellas para caracterizarlo. Todo proyecto independientemente de los inconvenientes que presente o su naturaleza podemos dividirlo en cuatro fases: análisis, diseño, desarrollo, cierre. Dependiendo del tipo de metodología, estas fases serán más cortas o largas, se repiten con más frecuencia o con menor frecuencia, se nominarán de otra manera, pero la estructura base se conserva.

Otra pregunta sería: ¿Y dónde quedan las otras fases cómo el monitoreo, la planeación, las pruebas, la implementación, control, inicio, la instalación, el despliegue, el mantenimiento, las lecciones aprendidas, el post mortem?, todas las anteriores o son sinónimo de alguna de las 4 o se pueden incluir dentro de ella.

Tabla 1. Fases del desarrollo de *software*

Análisis	Inicio
Diseño	Planeación
Desarrollo	Despliegue, implementación, pruebas, mantenimiento, control, monitoreo, codificación
Cierre	Instalación, lecciones aprendidas, post mortem

Fuente: elaboración propia

Otro cuestionamiento: ¿Qué tan importante es conocer cómo se programa y cuánto tiempo se invierte en cumplir a satisfacción un requerimiento de un cliente? A menos que programar sea un "*hobby*" esto debe ser importante, no solo porque permite conocer y poder hacerlo cada vez mejor si no porque también es importante para otros, quien hace bien el trabajo de seguro tendrá alguien que quiera que lo haga por él (cliente), si se quiere que hagan bien el trabajo, es necesario tener una forma de medir y controlar lo que están haciendo los colaboradores.

Cómo mejorar...

La disciplina es fundamental en el desarrollo de *software*, recuerde que codificar es un ejercicio, por tanto, es importante que lo practique a diario para ganar habilidades.

El *software* a diferencia de otro tipo de producto no se puede cuantificar debido a que no todas las personas programan de la misma manera, la forma como se resuelve un inconveniente no siempre es la misma debido a que no siempre se está resolviendo el mismo problema y porque hay muchos caminos para responder a la misma situación.

Existe una complejidad mayor a la hora de determinar cuánto tiempo tarda en construirse un *software*, que la que existe al tratar de determinar una tarea repetitiva o de industria como por ejemplo construir un muro, ya que la producción de *software* es considerada una obra literaria, de hecho, se registra como tal. *"La Oficina de Registro de la Dirección Nacional de Derecho de Autor, presta el servicio gratuito de registro de obras literarias y artísticas, entre ellas el soporte lógico o software"* (Ministerio del interior, 2017).

¿Por qué si el *software* es considerado como una obra de arte o literaria, se insiste en que este trabajo se puede medir, cuantificar, estandarizar y predecir?

Si bien no es posible llegar a una medida exacta si debemos tener una buena aproximación, de manera sistemática podemos establecer unos estándares de codificación y medición, que mejoran nuestra productividad y permite una correcta administración de nuestro esfuerzo. A lo largo del tiempo, ha habido esfuerzos por hacer esto posible. Es el caso de algunos autores como el profesor Watts Humphrey de la Universidad Carnegie Mellon quien como investigador tiempo completo del SEI, (Software Engineering Institute (SEI), 2017) desarrolló un conjunto de buenas prácticas y formatos que mejoran el proceso de construcción de *software* a partir del modelo de madurez CMMI aplicado a empresas. A estas buenas prácticas se les conoce como proceso personal de *software* o PSP por sus siglas en ingles. PSP muestra a los ingenieros cómo administrar la calidad desde el comienzo del trabajo, cómo analizar los resultados de cada trabajo y cómo usar los resultados para mejorar el proceso para el próximo proyecto. (Humphrey, 2002).

Al buscar respuesta a través de la historia de los interrogantes anteriormente mostrados y el hecho de enfrentarse al desarrollo de *software* ayuda a concluir que se debe medir y estandarizar para que la comunicación sea efectiva y en el proceso de construcción de *software* se garantice la calidad y se pueda mejorar de manera constante.

2. Métricas vs Indicadores de desarrollo de *software*

Medir es el primer paso para caracterizar y entender cómo es el proceso de construcción de *software*. Usualmente se confunde un indicador con una métrica por eso que es necesario definir ambos términos:

Una métrica es una característica que nos interesa medir, cuantificar para obtener una variable. Por ejemplo, el número de horas de trabajo o el número de líneas de código. Por si sola no nos dice mayor cosa, pero en conjunto puede servir de indicador.

Un indicador es el resultado final de utilizar una o más métricas para conseguir responder a una variable de interés, por ejemplo, el rendimiento (líneas de código por hora):

$$\text{Rendimiento (LOC/h)} = \frac{\text{Tiempo de trabajo en minutos}}{\text{Número de líneas de código} \times 60}$$

Figura 1. Formula de rendimiento

Fuente: elaboración propia

3. Formatos de registro y medición

Para este curso en particular inicialmente nos interesa medir los siguientes aspectos:

Fase: De acuerdo a la Tabla 1 selecciona la fase en la cual se está desarrollando el trabajo. Por ejemplo: "Codificación".

Descripción de la actividad: De manera simple, precisa y concreta se explica la actividad a realizar, por ejemplo: *"Codificar un tabla de 4*5 dónde se liste el proceso personal de software de los 12 registros de la base de datos".*

Tiempo Planeado: Se estima el tiempo en minutos de lo que considera se puede demorar realizando el trabajo *"120"*.

Tiempo Real: Cada vez que realice un aporte en tiempo al proceso, en cualquiera de las fases, se registra el tiempo utilizado para tal fin. Por ejemplo: "60 min" sin tener en cuenta las interrupciones.

Tiempo de Interrupción: 30 min.

Descripción de la interrupción: "Charla con un amig@."

Desfase: Es la diferencia en tiempo entre la estimación y el tiempo real sin tener en cuenta las interrupciones, este puede ser positivo, negativo o cero. Por ejemplo: "30".

Acumulado: Es la sumatoria de todos los desfases de las actividades y se calcula constantemente. Por ejemplo: "-60".

Tabla 2. Formato de registro proceso personal de *software*.

Fase	Nombre de la fase					
Descripción de la actividad	Tiempo Planeado	Tiempo Real	Tiempo de Interrupción	Descripción de la interrupción	Desfase (+/-)	Acumulado
Codificar un tabla de 4*5 dónde se liste el proceso personal de *software* de los 12 registros de la base de datos.	120	60	30	Charla con un amig@.	-60	-60

Fuente: elaboración propia

Es importante revisar otros formatos, discutir los propuestos en este apartado y crear sus propias versiones, sin embargo, la estandarización es fundamental para poder realizar comparaciones con otros programadores.

¿Qué información se puede obtener a través de la tabla? ¿Cómo puedo mejorar mi proceso de *software*?

Se puede obtener el tiempo **total de interrupción**, a través de la suma de cada una de las interrupciones de las actividades. Esta información es útil para analizar y plantear posibles planes de mejora y evitar que se presenten nuevamente esas interrupciones. Puede elaborar un listado de compromisos o estrategias de mejora, para implementar en futuros ejercicios de programación.

El tiempo de ***desfase*** o **desfase de estimación total**, es el resultante de sumar cada uno de los acumulados por actividad, con esa información es posible determinar qué tan acertada fue la estimación. También es un insumo para poder determinar cuál de todas las actividades tuvo mayor impacto en el desfase.

Para poder comparar los datos con algún compañero es necesario que el proyecto sea el mismo, no necesariamente la misma distribución de fases o actividades. Además debe totalizar los tiempos de interrupciones y realizar un análisis con y sin esta información.

Una vez completado el registro y el proceso de auto evaluación, deberá estar en capacidad de responder los siguientes interrogantes:

¿Qué dato podría adicionar al formato?, ¿Cuál no es necesario y por qué? Haga un proceso de auto evaluación y plantee un proceso de mejora para la próxima iteración. Compare sus resultados con 2 de sus compañeros y concluya.

En síntesis...

El proceso personal de *software*: implica conciencia sobre un conjunto de características que debemos tener en cuenta, durante todas las fases de desarrollo de proyectos de *software*.
La métrica: hace parte del Indicador.
El Rendimiento: está asociado a la cantidad de líneas de código sin defectos y basado en un estándar y que producen un determinado tiempo, usualmente en horas.

Referencias bibliográficas

Humphrey, W. S. (2002). Personal Software Process (PSP). PERSONAL SOFTWARE PROCESS (PSP). *Encyclopedia Of Software Engineering, Volume 2*, 948-961.

Ministerio del interior. (01 de 05 de 2017). *Dirección Nacional de Derechos de Autor, unidad administrativa especial.* Obtenido de Registro de soporte lógico (software): http://derechodeautor.gov.co/software

Software Engineering Institute (SEI). (01 de 05 de 2017). *Software Engineering Institute.* Obtenido de Carnegie Mellon University: http://www.sei.cmu.edu/about/

W3C. (05 de 02 de 2017). *Markup Validation Service.* Obtenido de https://validator.w3.org/

INFORMACIÓN TÉCNICA

FACULTAD DE
INGENIERÍA, DISEÑO
E INNOVACIÓN

Módulo: Proceso de *Software* Personal PSP

Unidad 1: Introducción al proceso de *software* personal

Escenario 1: Características fundamentales de los procesos de desarrollo de *software*

Autor: Diego Iván Oliveros Acosta

Asesor Pedagógico: Jeiner Velandia

Diseñador Gráfico: Kelly Yohana Valencia Forero

Asistente: Ginna Paola Quiroga Espinosa

Este material pertenece al Politécnico Grancolombiano. Por ende, es de uso exclusivo de las Instituciones adscritas a la Red Ilumno. Prohibida su reproducción total o parcial.